

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('/content/bbc_data.csv')
```

```
df.head()
```

		data	labels	
0	Musicians to tackle US red tape	Musicians gro...	entertainment	
1	U2s desire to be number one	U2, who have won ...	entertainment	
2	Rocker Doherty in on-stage fight	Rock singer ...	entertainment	
3	Snicket tops US box office chart	The film ada...	entertainment	
4	Oceans Twelve raids box office	Oceans Twelve,...	entertainment	

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
df.shape
```

```
(2225, 2)
```

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
import re
```

```
df['labels'].value_counts()
```

	count
labels	
sport	511
business	510
politics	417
tech	401
entertainment	386

```
dtype: int64
```

1. Preprocessing

```
def clean_text(text):
    """
    Comprehensive text preprocessing:
    - Remove HTML tags
    - Remove punctuation and numbers
    - Lowercase conversion
    - Tokenization
    - Stopwords removal
    - Lemmatization
    """
    if not isinstance(text, str):
        return ""

    # Remove HTML tags
    text = re.sub(r'<.*?>', '', text)

    # Remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)

    # Remove email addresses
    text = re.sub(r'\S+@\S+', '', text)

    # Remove punctuation and numbers
    text = re.sub(r'^a-zA-Z]', ' ', text)

    # Lowercase
    text = text.lower()

    # Tokenization
    tokens = word_tokenize(text)

    # Remove stopwords and short words
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words and len(word) > 3]

    # Lemmatization
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]

    # Join tokens back
    cleaned_text = " ".join(tokens)

    # Remove extra spaces
    cleaned_text = re.sub(' +', ' ', cleaned_text).strip()

    return cleaned_text


# Download NLTK resources
nltk.download('punkt', quiet=True)
nltk.download('wordnet', quiet=True)
```

```
nltk.download('stopwords', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
```

```
True
```

```
# Apply preprocessing
import nltk
nltk.download('punkt_tab')
df['clean_text'] = df['data'].apply(clean_text)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

```
print(f"Original: {df['data'].iloc[0][:200]}...")
print(f"\nCleaned: {df['clean_text'].iloc[0][:200]}...")
```

```
Original: Musicians to tackle US red tape Musicians groups are to tackle US
Cleaned: musician tackle red tape musician group tackle visa regulation blame
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
#Task 2 {TF-IDF}Converting the cleaned text to Numerical Vectors
tfidf = TfidfVectorizer(
    max_df=0.85,          # ignore terms that appear in >85% of documents
    min_df=3,             # ignore terms that appear in <3 documents
    max_features=10000,   # limit vocabulary size
    ngram_range=(1,2),    # unigrams + bigrams
    sublinear_tf=True,    # use log scaling for term frequency
    norm='l2'             # L2 normalization
)

X_tfidf = tfidf.fit_transform(df['clean_text'])
print(f" TF-IDF vectorization complete")
print(f"  Shape: {X_tfidf.shape}")
print(f"  Vocabulary size: {len(tfidf.get_feature_names_out())}")
print(f"  Sparsity: {(1.0 - X_tfidf.nnz / (X_tfidf.shape[0] * X_tfidf.shape[1]))}")
```

```
TF-IDF vectorization complete
Shape: (2225, 10000)
Vocabulary size: 10000
Sparsity: 0.9850
```

```
#Dimensionality Reduction
from sklearn.decomposition import TruncatedSVD, PCA
from sklearn.preprocessing import Normalizer, StandardScaler
from sklearn.pipeline import make_pipeline
```

```
n_components = 100
svd = TruncatedSVD(n_components=n_components, random_state=42)
normalizer = Normalizer(copy=False)
lsa = make_pipeline(svd, normalizer)
```

```
X_lsa = lsa.fit_transform(X_tfidf)
```

```
# Task 3 Applying Clustering Algorithm
```

```
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
```

```
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davie
```

```
#finding optimal number of clusters
```

```
silhouette_scores = []
```

```
calinski_scores = []
```

```
davies_bouldin_scores = []
```

```
inertias = []
```

```
K_range = range(3, 11)
```

```
for k in K_range:
```

```
    kmeans_temp = KMeans(
```

```
        n_clusters=k,
```

```
        init='k-means++',
```

```
        max_iter=300,
```

```
        n_init=20,
```

```
        random_state=42
```

```
    )
```

```
    labels_temp = kmeans_temp.fit_predict(X_lsa)
```

```
    sil = silhouette_score(X_lsa, labels_temp)
```

```
    cal = calinski_harabasz_score(X_lsa, labels_temp)
```

```
    dav = davies_bouldin_score(X_lsa, labels_temp)
```

```
    silhouette_scores.append(sil)
```

```
    calinski_scores.append(cal)
```

```
    davies_bouldin_scores.append(dav)
```

```
    inertias.append(kmeans_temp.inertia_)
```

```
    print(f"k={k}: Silhouette={sil:.4f}, Calinski-Harabasz={cal:.2f}, Davies
```

```
k=3: Silhouette=0.0550, Calinski-Harabasz=91.63, Davies-Bouldin=4.2402
```

```
k=4: Silhouette=0.0650, Calinski-Harabasz=90.27, Davies-Bouldin=3.6281
```

```
k=5: Silhouette=0.0769, Calinski-Harabasz=88.12, Davies-Bouldin=3.3480
```

```
k=6: Silhouette=0.0810, Calinski-Harabasz=81.26, Davies-Bouldin=3.3015
```

```
k=7: Silhouette=0.0831, Calinski-Harabasz=75.62, Davies-Bouldin=3.2039
```

```
k=8: Silhouette=0.0853, Calinski-Harabasz=71.02, Davies-Bouldin=3.0390
```

```
k=9: Silhouette=0.0907, Calinski-Harabasz=68.99, Davies-Bouldin=2.9226
```

```
k=10: Silhouette=0.0860, Calinski-Harabasz=66.62, Davies-Bouldin=3.1067
```

```
# Plot evaluation metrics
```

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

```
axes[0, 0].plot(K_range, inertias, 'bo-', linewidth=2, markersize=8)
```

```
axes[0, 0].set_xlabel('Number of Clusters (k)', fontsize=11)
```

```
axes[0, 0].set_ylabel('Inertia', fontsize=11)
```

```
axes[0, 0].set_title('Elbow Method', fontsize=12, fontweight='bold')
```

```
axes[0, 0].grid(True, alpha=0.3)
```

```
axes[0, 1].plot(K_range, silhouette_scores, 'ro-', linewidth=2, markersize=8)
```

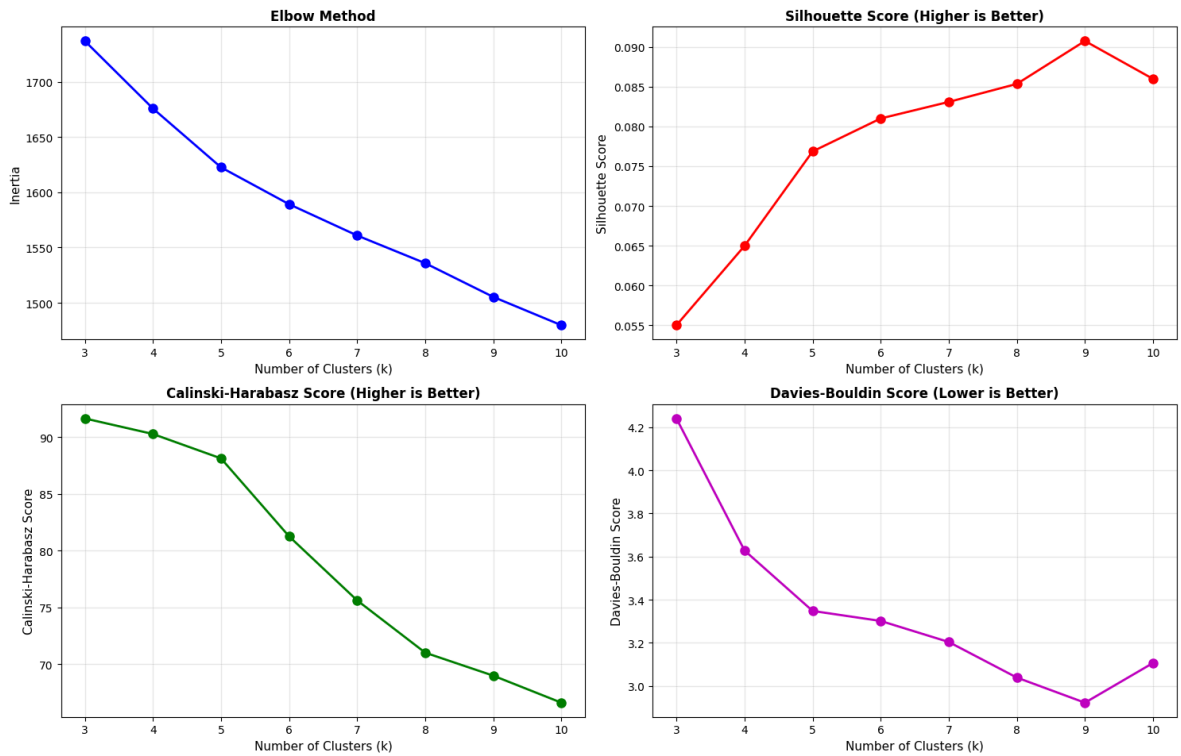
```
axes[0, 1].set_xlabel('Number of Clusters (k)', fontsize=11)
axes[0, 1].set_ylabel('Silhouette Score', fontsize=11)
axes[0, 1].set_title('Silhouette Score (Higher is Better)', fontsize=12, font
axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].plot(K_range, calinski_scores, 'go-', linewidth=2, markersize=8)
axes[1, 0].set_xlabel('Number of Clusters (k)', fontsize=11)
axes[1, 0].set_ylabel('Calinski-Harabasz Score', fontsize=11)
axes[1, 0].set_title('Calinski-Harabasz Score (Higher is Better)', fontsize=
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].plot(K_range, davies_bouldin_scores, 'mo-', linewidth=2, markersi
axes[1, 1].set_xlabel('Number of Clusters (k)', fontsize=11)
axes[1, 1].set_ylabel('Davies-Bouldin Score', fontsize=11)
axes[1, 1].set_title('Davies-Bouldin Score (Lower is Better)', fontsize=12,
axes[1, 1].grid(True, alpha=0.3)

plt.suptitle('K-Means Clustering: Optimal K Selection', fontsize=14, fontwei
plt.tight_layout()
plt.show()
```

K-Means Clustering: Optimal K Selection



```
# Select optimal k
optimal_k = K_range[np.argmax(silhouette_scores)]
print(f"\n✓ Optimal k selected: {optimal_k}")

# Final K-Means clustering
kmeans = KMeans(
    n_clusters=optimal_k,
    init='k-means++',
    max_iter=300,
    n_init=20,
    random_state=42
)
df['kmeans_cluster'] = kmeans.fit_predict(X_lsa)
```

```

kmeans_sil = silhouette_score(X_lsa, df['kmeans_cluster'])
kmeans_cal = calinski_harabasz_score(X_lsa, df['kmeans_cluster'])
kmeans_dav = davies_bouldin_score(X_lsa, df['kmeans_cluster'])

print(f"\nK-Means Results (k={optimal_k}):")
print(f"  Silhouette Score: {kmeans_sil:.4f}")
print(f"  Calinski-Harabasz Score: {kmeans_cal:.2f}")
print(f"  Davies-Bouldin Score: {kmeans_dav:.4f}")
print(f"\nCluster distribution:")
print(df['kmeans_cluster'].value_counts().sort_index())

```

✓ Optimal k selected: 9

K-Means Results (k=9):
 Silhouette Score: 0.0907
 Calinski-Harabasz Score: 68.99
 Davies-Bouldin Score: 2.9226

Cluster distribution:
 kmeans_cluster

0	333
1	413
2	424
3	95
4	329
5	166
6	238
7	138
8	89

Name: count, dtype: int64

```

# 3.2: HIERARCHICAL CLUSTERING
# -----
print(f"\n--- HIERARCHICAL CLUSTERING ---")

hierarchical = AgglomerativeClustering(
    n_clusters=optimal_k,
    linkage='ward'
)
df['hierarchical_cluster'] = hierarchical.fit_predict(X_lsa)

hier_sil = silhouette_score(X_lsa, df['hierarchical_cluster'])
hier_cal = calinski_harabasz_score(X_lsa, df['hierarchical_cluster'])
hier_dav = davies_bouldin_score(X_lsa, df['hierarchical_cluster'])

print(f"Hierarchical Clustering Results (k={optimal_k}):")
print(f"  Silhouette Score: {hier_sil:.4f}")
print(f"  Calinski-Harabasz Score: {hier_cal:.2f}")
print(f"  Davies-Bouldin Score: {hier_dav:.4f}")
print(f"\nCluster distribution:")
print(df['hierarchical_cluster'].value_counts().sort_index())

```

[Show hidden output](#)

 Start coding or [generate](#) with AI.

```
# 3.3: DBSCAN CLUSTERING
# -----
print(f"\n--- DBSCAN CLUSTERING ---")

# DBSCAN requires careful parameter tuning
# We'll test different eps values
from sklearn.neighbors import NearestNeighbors

# Find optimal eps using k-distance graph
neighbors = NearestNeighbors(n_neighbors=5)
neighbors_fit = neighbors.fit(X_lsa)
distances, indices = neighbors_fit.kneighbors(X_lsa)
distances = np.sort(distances[:, -1], axis=0)

# Try multiple eps values
eps_values = [0.5, 1.0, 1.5, 2.0, 2.5]
best_dbscan_score = -1
best_eps = eps_values[0]

print("Testing different eps values...")
for eps in eps_values:
    dbscan_temp = DBSCAN(eps=eps, min_samples=5)
    labels_temp = dbscan_temp.fit_predict(X_lsa)

    n_clusters = len(set(labels_temp)) - (1 if -1 in labels_temp else 0)
    n_noise = list(labels_temp).count(-1)

    if n_clusters > 1: # Need at least 2 clusters for silhouette score
        try:
            score = silhouette_score(X_lsa, labels_temp)
            print(f"  eps={eps}: clusters={n_clusters}, noise={n_noise}, sil

            if score > best_dbscan_score:
                best_dbscan_score = score
                best_eps = eps
        except:
            print(f"  eps={eps}: clusters={n_clusters}, noise={n_noise}, sil
    else:
        print(f"  eps={eps}: clusters={n_clusters}, noise={n_noise} (too few

--- DBSCAN CLUSTERING ---
Testing different eps values...
eps=0.5: clusters=46, noise=1815, silhouette=-0.0709
eps=1.0: clusters=1, noise=2 (too few clusters)
eps=1.5: clusters=1, noise=0 (too few clusters)
eps=2.0: clusters=1, noise=0 (too few clusters)
eps=2.5: clusters=1, noise=0 (too few clusters)
```

Start coding or [generate](#) with AI.

```
# Final DBSCAN with best eps
dbscan = DBSCAN(eps=best_eps, min_samples=5)
df['dbscan_cluster'] = dbscan.fit_predict(X_lsa)

n_clusters_dbscan = len(set(df['dbscan_cluster'])) - (1 if -1 in df['dbscan_
n_noise_dbscan = list(df['dbscan_cluster']).count(-1)

print(f"\nDBSCAN Results (eps={best_eps}):")
print(f"  Number of clusters: {n_clusters_dbscan}")
print(f"  Noise points: {n_noise_dbscan}")

if n_clusters_dbscan > 1:
    # Calculate silhouette score excluding noise points
    mask = df['dbscan_cluster'] != -1
    if sum(mask) > 0:
        dbscan_sil = silhouette_score(X_lsa[mask], df['dbscan_cluster'][mask])
        print(f"  Silhouette Score (excl. noise): {dbscan_sil:.4f}")

print(f"\nCluster distribution:")
print(df['dbscan_cluster'].value_counts().sort_index())
```

```
DBSCAN Results (eps=0.5):
  Number of clusters: 47
  Noise points: 1815
  Silhouette Score (excl. noise): 0.5614
```

```
Cluster distribution:
dbscan_cluster
```

```
-1      1815
0         10
1         10
2         29
3          5
4          5
5          6
6          8
7         10
8         22
9          9
10         8
11         5
12         7
13         5
14         8
15         5
16         6
17        14
18         7
19         9
20        30
21         6
22         9
23        10
24         5
```

```

25      5
26      5
27      5
28      6
29      5
30     12
31      6
32     11
33     15
34      8
35      8
36      7
37      5
38      5
39      8
40     10
41      7
42     13
43      7
44      8
45      6

```

Name: count, dtype: int64

Start coding or [generate](#) with AI.

```

# COMPARISON OF ALGORITHMS
# -----
print(f"\n{'='*70}")
print(f"CLUSTERING ALGORITHMS COMPARISON")
print(f"{'='*70}")
print(f"{'Algorithm':<20} {'Silhouette':<15} {'Calinski-H':<15} {'Davies-B':<15}")
print(f"{'-'*70}")
print(f"{'K-Means':<20} {kmeans_sil:<15.4f} {kmeans_cal:<15.2f} {kmeans_dav:<15.2f}")
print(f"{'Hierarchical':<20} {hier_sil:<15.4f} {hier_cal:<15.2f} {hier_dav:<15.2f}")
if n_clusters_dbscan > 1 and sum(mask) > 0:
    print(f"{'DBSCAN':<20} {dbscan_sil:<15.4f} {'N/A':<15} {'N/A':<15}")
else:
    print(f"{'DBSCAN':<20} {'N/A':<15} {'N/A':<15} {'N/A':<15}")

# Use K-Means for further analysis (typically performs best)
df['cluster'] = df['kmeans_cluster']

```

```

=====
CLUSTERING ALGORITHMS COMPARISON
=====

```

Algorithm	Silhouette	Calinski-H	Davies-B
K-Means	0.0907	68.99	2.9226
Hierarchical	0.0833	63.55	3.0053
DBSCAN	0.5614	N/A	N/A

Start coding or [generate](#) with AI.

```

# TASK 4: VISUALIZE CLUSTERS (2D t-SNE plots)
# =====
print(f"\n{' '*70}")
print(f"TASK 4: CLUSTER VISUALIZATION (t-SNE)")
print(f"{' '*70}")

# Use subset if dataset is large
n_samples = min(len(df), 2000)
if len(df) > n_samples:
    indices = np.random.choice(len(df), n_samples, replace=False)
    X_vis = X_lsa[indices]
    kmeans_vis = df['kmeans_cluster'].iloc[indices]
    hier_vis = df['hierarchical_cluster'].iloc[indices]
    dbscan_vis = df['dbscan_cluster'].iloc[indices]
    labels_vis = df['labels'].iloc[indices] if 'labels' in df.columns else None
else:
    X_vis = X_lsa
    kmeans_vis = df['kmeans_cluster']
    hier_vis = df['hierarchical_cluster']
    dbscan_vis = df['dbscan_cluster']
    labels_vis = df['labels'] if 'labels' in df.columns else None

print("Computing t-SNE projection...")
tsne = TSNE(
    n_components=2,
    random_state=42,
    perplexity=30,
    n_iter=1000,
    learning_rate=200
)
X_tsne = tsne.fit_transform(X_vis)
print("✓ t-SNE computation complete")

# Create comprehensive visualization
fig = plt.figure(figsize=(18, 12))

# K-Means t-SNE
ax1 = plt.subplot(2, 3, 1)
scatter1 = ax1.scatter(X_tsne[:, 0], X_tsne[:, 1], c=kmeans_vis, cmap='tab10',
                      alpha=0.6, s=50, edgecolors='w', linewidth=0.5)
ax1.set_title(f'K-Means Clustering\n(Silhouette: {kmeans_sil:.4f})',
              fontsize=12, fontweight='bold')
ax1.set_xlabel('t-SNE Component 1')
ax1.set_ylabel('t-SNE Component 2')
plt.colorbar(scatter1, ax=ax1, label='Cluster')
ax1.grid(True, alpha=0.3)

# Hierarchical t-SNE
ax2 = plt.subplot(2, 3, 2)
scatter2 = ax2.scatter(X_tsne[:, 0], X_tsne[:, 1], c=hier_vis, cmap='tab10',
                      alpha=0.6, s=50, edgecolors='w', linewidth=0.5)
ax2.set_title(f'Hierarchical Clustering\n(Silhouette: {hier_sil:.4f})',
              fontsize=12, fontweight='bold')
ax2.set_xlabel('t-SNE Component 1')
ax2.set_ylabel('t-SNE Component 2')

```

```

plt.colorbar(scatter2, ax=ax2, label='Cluster')
ax2.grid(True, alpha=0.3)

# DBSCAN t-SNE
ax3 = plt.subplot(2, 3, 3)
scatter3 = ax3.scatter(X_tsne[:, 0], X_tsne[:, 1], c=dbscan_vis, cmap='tab10',
                      alpha=0.6, s=50, edgecolors='w', linewidth=0.5)
if n_clusters_dbscan > 1 and sum(mask) > 0:
    title = f'DBSCAN Clustering\n(Silhouette: {dbscan_sil:.4f})'
else:
    title = f'DBSCAN Clustering\n(Clusters: {n_clusters_dbscan})'
ax3.set_title(title, fontsize=12, fontweight='bold')
ax3.set_xlabel('t-SNE Component 1')
ax3.set_ylabel('t-SNE Component 2')
plt.colorbar(scatter3, ax=ax3, label='Cluster')
ax3.grid(True, alpha=0.3)

# True labels (if available)
if labels_vis is not None:
    ax4 = plt.subplot(2, 3, 4)
    # Convert labels to numeric for plotting
    label_mapping = {label: idx for idx, label in enumerate(labels_vis.unique())}
    numeric_labels = labels_vis.map(label_mapping)

    scatter4 = ax4.scatter(X_tsne[:, 0], X_tsne[:, 1], c=numeric_labels, cmap='tab10',
                          alpha=0.6, s=50, edgecolors='w', linewidth=0.5)
    ax4.set_title('True Labels', fontsize=12, fontweight='bold')
    ax4.set_xlabel('t-SNE Component 1')
    ax4.set_ylabel('t-SNE Component 2')
    cbar = plt.colorbar(scatter4, ax=ax4)
    cbar.set_label('Label')
    ax4.grid(True, alpha=0.3)

# Cluster size comparison
ax5 = plt.subplot(2, 3, 5)
cluster_sizes = df['kmeans_cluster'].value_counts().sort_index()
ax5.bar(range(len(cluster_sizes)), cluster_sizes.values, color='steelblue')
ax5.set_xlabel('Cluster ID')
ax5.set_ylabel('Number of Documents')
ax5.set_title('K-Means Cluster Sizes', fontsize=12, fontweight='bold')
ax5.grid(True, alpha=0.3, axis='y')

# True label distribution
ax6 = plt.subplot(2, 3, 6)
label_counts = df['labels'].value_counts()
ax6.bar(range(len(label_counts)), label_counts.values, color='coral', align='center')
ax6.set_xticks(range(len(label_counts)))
ax6.set_xticklabels(label_counts.index, rotation=45, ha='right')
ax6.set_ylabel('Number of Documents')
ax6.set_title('True Label Distribution', fontsize=12, fontweight='bold')
ax6.grid(True, alpha=0.3, axis='y')

plt.suptitle('Document Clustering Visualization', fontsize=16, fontweight='bold')
plt.tight_layout()

```

```
plt.show()
```

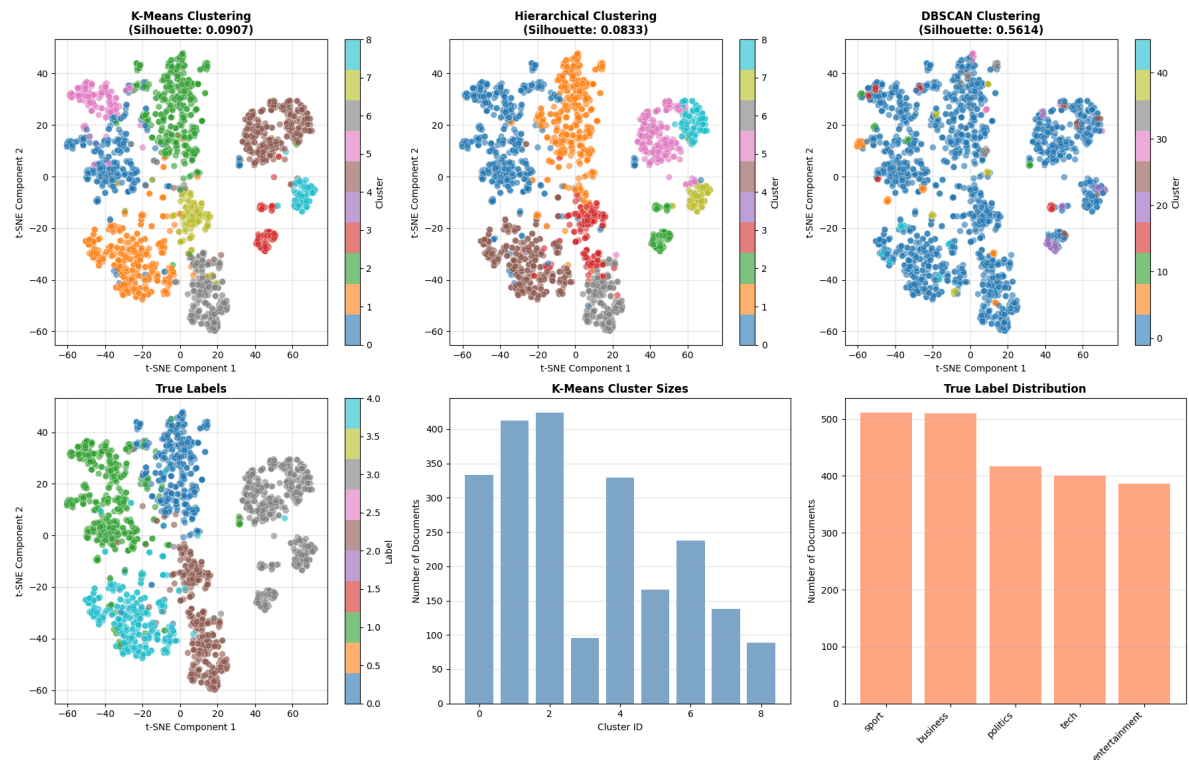
TASK 4: CLUSTER VISUALIZATION (t-SNE)

Computing t-SNE projection...

/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning::warn(

✓ t-SNE computation complete

Document Clustering Visualization



TASK 5: IDENTIFY MAIN TOPICS/KEYWORDS IN EACH CLUSTER

```
# =====
print(f"\n{' '*70}")
```

```

print(f"TASK 5: TOPIC IDENTIFICATION & KEYWORD EXTRACTION")
print(f"{'='*70}")

def get_top_keywords_per_cluster(n_terms=15):
    """Extract top keywords for each cluster"""

    # Get cluster centroids in original TF-IDF space
    original_space_centroids = svd.inverse_transform(kmeans.cluster_centers_)
    order_centroids = original_space_centroids.argsort()[:, :-1]
    terms = tfidf.get_feature_names_out()

    cluster_topics = {}

    for i in range(optimal_k):
        print(f"\n{'='*70}")
        print(f"CLUSTER {i}")
        print(f"{'='*70}")

        # Get top terms
        top_terms = [terms[ind] for ind in order_centroids[i, :n_terms]]
        cluster_topics[i] = top_terms

        print(f"Size: {sum(df['cluster'] == i)} documents")
        print(f"\nTop Keywords:")
        for idx, term in enumerate(top_terms, 1):
            print(f" {idx:2d}. {term}")

        # Show true label distribution for this cluster
        if 'labels' in df.columns:
            cluster_labels = df[df['cluster'] == i]['labels'].value_counts()
            print(f"\nTrue Label Distribution:")
            for label, count in cluster_labels.items():
                percentage = (count / sum(df['cluster'] == i)) * 100
                print(f" {label}: {count} ({percentage:.1f}%)")

        # Show sample documents
        print(f"\nSample Documents:")
        cluster_docs = df[df['cluster'] == i]
        for idx, (_, row) in enumerate(cluster_docs.head(2).iterrows(), 1):
            print(f"\n Document {idx}:")
            if 'labels' in df.columns:
                print(f" Label: {row['labels']}")
            print(f" Text: {row['data'][:250]}...")

    return cluster_topics

cluster_keywords = get_top_keywords_per_cluster(n_terms=20)

# -----
# VISUALIZE KEYWORDS
# -----
print(f"\n{'='*70}")
print(f"KEYWORD VISUALIZATION")
print(f"{'='*70}")

```

```
# Create word cloud style visualization
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for i in range(min(optimal_k, 6)): # Show up to 6 clusters
    if i < optimal_k:
        keywords = cluster_keywords[i][:10]
        # Create simple bar chart of top keywords
        axes[i].barh(range(len(keywords)), range(len(keywords), 0, -1), color=axes[i].set_yticks(range(len(keywords)))
        axes[i].set_yticklabels(keywords, fontsize=9)
        axes[i].set_xlabel('Importance Rank')
        axes[i].set_title(f'Cluster {i} - Top Keywords\n({sum(df["cluster"]
                                fontsize=11, fontweight='bold')
        axes[i].invert_xaxis()
        axes[i].grid(True, alpha=0.3, axis='x')
    else:
        axes[i].axis('off')

plt.tight_layout()
plt.show()
```

[Show hidden output](#)

```
# CLUSTER-LABEL CONFUSION MATRIX
# -----
if 'labels' in df.columns:
    print(f"\n{'='*70}")
    print(f"CLUSTER vs TRUE LABEL ANALYSIS")
    print(f"{'='*70}")

    # Create confusion matrix
    confusion_matrix = pd.crosstab(
        df['kmeans_cluster'],
        df['labels'],
        rownames=['Cluster'],
        colnames=['True Label']
    )

    print("\nConfusion Matrix:")
    print(confusion_matrix)

    # Visualize
    plt.figure(figsize=(10, 8))
    sns.heatmap(
        confusion_matrix,
        annot=True,
        fmt='d',
        cmap='YlOrRd',
        cbar_kws={'label': 'Number of Documents'}
    )
    plt.title('K-Means Clusters vs True Labels', fontsize=14, fontweight='b')
    plt.xlabel('True Labels', fontsize=12)
```

```
plt.ylabel('Clusters', fontsize=12)
plt.tight_layout()
...

```

=====

CLUSTER vs TRUE LABEL ANALYSIS

=====

Confusion Matrix:

True Label	business	entertainment	politics	sport	tech
Cluster					
0	322	0	5	1	5
1	10	11	6	0	386
2	13	5	404	1	1
3	0	1	0	94	0
4	0	0	1	326	2
5	164	1	1	0	0
6	0	234	0	0	4
7	1	134	0	0	3
8	0	0	0	89	0

